基于 PPO 算法和自注意力机制的 Pong 游戏智能体软件 v1.0

用户手册

目录

第一章　简介 3
1. 背景 3
2. 简介 3
3. 目标用户 3
4. 主要功能 3
5. 优势与创新 4
第二章　运行环境 4
1. 硬件环境 5
2. 软件环境 5
第三章　主要功能（用户端） 6
1. 模型训练 6
2. 模型测试 6
3. 继续训练 7
4. 训练后测试 8
第四章 主要功能（开发人员/技术特性） 9
1. 网络架构：CNN + 多头空间自注意力 9
2. PPO 算法实现 9
3. 环境处理与帧堆叠 10
4. 可配置参数与扩展性 10
5. 注意力机制可视化（可选） 10
第五章 使用说明 11
1. 命令行参数 11
2. 运行示例 11
第六章 软件架构 12
第七章 输出文件与日志 13

第一章　简介

1. 背景

在强化学习研究中，Atari 游戏（如 Pong）常作为算法验证的基准环境。传统方法通常采用卷积神经网络提取图像特征，但在处理需要关注关键区域（如球、球拍）的任务时，注意力机制能够显著提升性能。本软件将 PPO（近端策略优化）算法与多头空间自注意力机制结合，旨在提供一个高效、可扩展的 Pong 游戏智能体训练与测试平台。

2. 简介

本软件是一个基于近端策略优化（PPO）算法与多头空间自注意力机制的强化学习智能体，用于训练和测试 Atari Pong 游戏。软件采用单文件模块化设计（combined_code.py），集成了 CNN 特征提取、空间注意力模块、Actor-Critic 架构等核心技术，支持从零训练、继续训练、模型测试等多种工作流，并提供 TensorBoard 实时监控训练过程。

3. 目标用户

- 强化学习研究人员：验证 PPO 与注意力机制的结合效果。
- 游戏 AI 开发者：快速搭建 Pong 游戏的智能体原型。

4. 主要功能

- 模型训练：支持 PPO 算法的完整训练流程，自动保存最佳模型。
- 模型测试：可加载训练好的模型进行可视化测试，实时显示游戏画面与累计奖励。
- 继续训练：支持从已有检查点恢复训练，便于长期实验。
- 训练后测试：训练完成后自动执行测试回合，评估模型性能。
- 日志记录：使用 TensorBoard 记录奖励、回合长度、FPS 等关键指标。

5. 优势与创新

- 注意力增强：引入多头空间自注意力模块，使智能体聚焦于游戏中的关键区域（球、球拍），提升特征提取效率。
- 单文件模块化：所有代码集成于单一文件，便于移植、修改和分发。
- 可视化训练监控：通过 TensorBoard 实时监控训练曲线，支持自定义对齐步数。
- 灵活配置：提供丰富的命令行参数，可调整训练步数、测试回合数、渲染速度等。
- 动画流畅：在测试模式中，游戏画面渲染延迟可调

第二章　运行环境

1. 硬件环境

- 最低配置：任意支持 Python 的 PC（CPU 模式）
- 推荐配置：NVIDIA GPU（CUDA 支持），以加速训练
- 外设要求：无特殊硬件需求，所有操作在 PC 上完成

2. 软件环境

- 操作系统：Windows 10/11、Linux（Ubuntu 18.04+）、macOS
- Python 版本：3.8 及以上
- 核心依赖库：
  - PyTorch ≥ 2.0
  - Gymnasium (Atari 环境) ≥ 0.28
  - Stable-Baselines3 ≥ 2.0
  - TensorBoard
  - ale-py (Arcade Learning Environment)
- 开发环境：任意支持 Python 的 IDE（如 VS Code、PyCharm）

第三章　主要功能（用户端）

1. 模型训练

用户可通过简单命令启动训练，系统会自动创建日志目录并保存模型。训练过程中，终端会输出当前回合的平均奖励、FPS 等信息（如图 1 所示）。



图 1 训练过程中终端输出的实时指标

2. 模型测试

测试模式下，系统加载指定模型（最新或最佳），并在可视化窗口中运行游戏。测试过程中实时显示步数和累计奖励（如图 2 所示）。

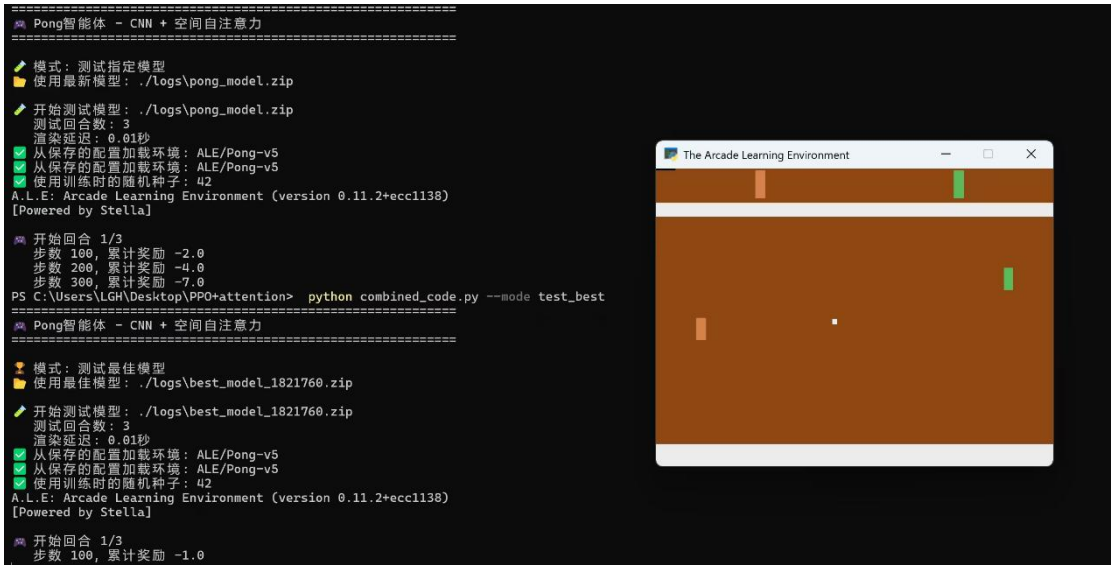


图 2 测试指定模型时的输出（步数 100，累计奖励-2.0）

3. 继续训练

若训练中断，用户可使用继续训练功能加载已有模型，并指定额外训练步数。系统会自动延续之前的日志记录，确保训练曲线连续（如图 3 所示）。

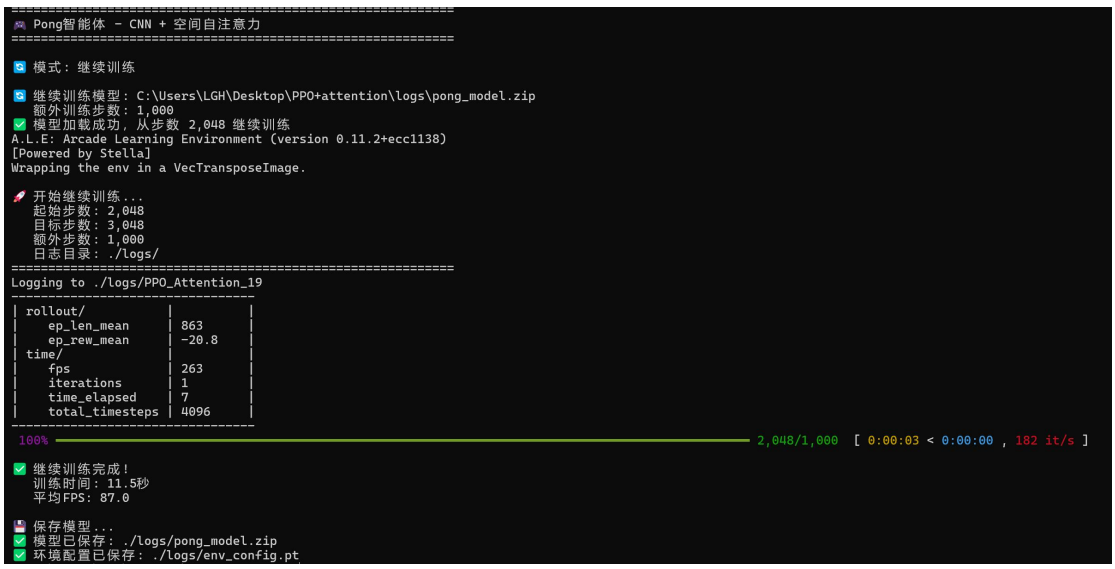


图 3 继续训练功能演示

4. 训练后测试

“训练并测试”模式允许用户在训练完成后立即进行若干回合的测试，快速评估刚训练出的模型性能。

5. 日志与监控

所有训练过程均记录在 TensorBoard 中，用户可运行 tensorboard --logdir ./logs 查看详细曲线，包括奖励变化、回合长度、损失函数等。

第四章 主要功能（开发人员/技术特性）

1. 网络架构：CNN + 多头空间自注意力

本软件的核心创新在于特征提取网络的设计，其结构如图 4 所示：

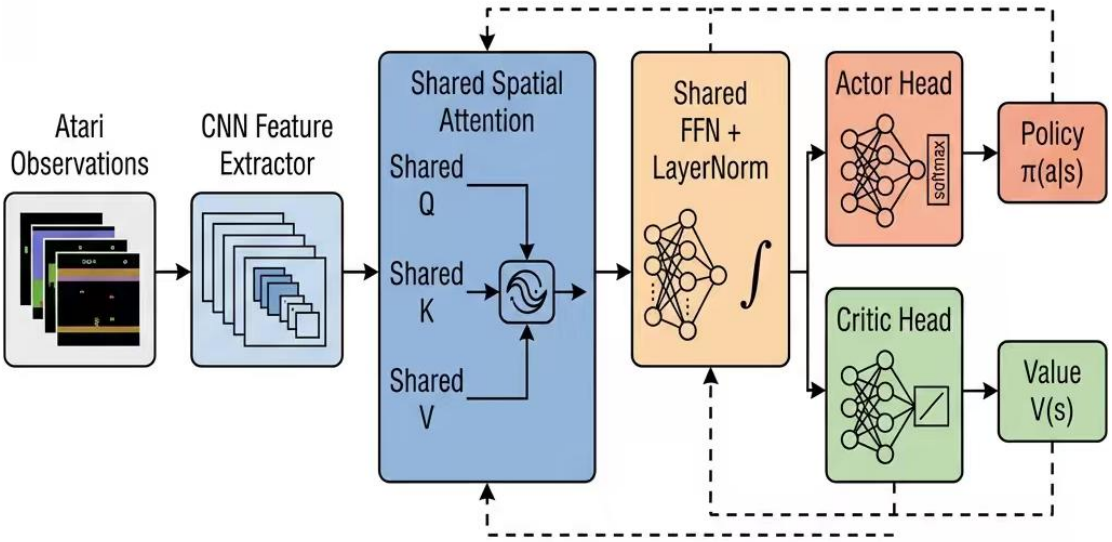


图 4 共享特征提取器+多头空间自注意力+ Actor-Critic 头

- **CNN 特征提取**：接收 84×84 的灰度图像（4 帧堆叠），通过三层卷积提取空间特征。
- **多头空间自注意力**：将 CNN 输出的特征图视为空间位置序列，计算自注意力，使模型关注球、球拍等关键区域。
- **Actor-Critic 头**：分别输出动作概率（策略）和状态价值（价值函数）。

2. PPO 算法实现

基于 Stable-Baselines3 的 PPO 实现，并自定义了策略网络以集成注

意力模块。关键参数（如学习率、GAE 系数、裁剪范围）均可通过修改配置类调整。

3. 环境处理与帧堆叠

- **图像预处理**：将 RGB 图像转为灰度、缩放至 84×84，并堆叠连续 4 帧作为状态输入，捕捉运动信息。
- **帧跳跃**：每 4 帧执行一次动作，提高训练效率。

4. 可配置参数与扩展性

- 注意力头数（默认 4）可在配置中修改。
- 支持更换其他 Atari 游戏（需调整动作空间）。
- 可通过继承 CnnAttentionFeaturesExtractor 类扩展其他注意力变体。

5. 注意力机制可视化

开发人员可扩展代码，将注意力权重可视化，观察智能体在游戏中的关注区域（需额外实现）。

第五章 使用说明

1. 命令行参数

参数	说明	默认值
--mode	运行模式（train/test/test_best/train_and_test/continue_train）	train
--model	模型路径（测试或继续训练时使用）	none
--steps	训练步数	200,000
--test_episodes	测试回合数	3
--render_delay	渲染延迟（秒）	0.01

2. 运行示例

```
# 训练 200 万步
python combined_code.py --mode train --steps 2000000

# 测试最新模型 5 回合
python combined_code.py --mode test --test_episodes 5

# 测试最佳模型
python combined_code.py --mode test_best

# 继续训练 5000 步
python combined_code.py --mode continue_train --model ./logs/pong_model.zip --steps 5000
```

第六章 软件架构

combined_code.py（单文件模块化设计）

Config	# 配置类
工具函数	# set_seed, normalize, create_directory
Preprocessor	# 预处理器类
PongWrapper	# 环境包装类
create_env	# 环境创建函数
create_test_env	# 测试环境创建函数
MultiHeadSpatialAttention	# 多头空间自注意力
SpatialAttentionBlock	# 空间注意力块
CnnAttentionFeaturesExtractor	# CNN+注意力特征提取器
CnnAttentionPolicy	# Actor-Critic 策略网络
SaveBestModelCallback	# 最佳模型保存回调
TensorboardStepAlignmentCallback	# TensorBoard 日志回调
PPOAlgorithm	# PPO 算法类
PongAgent	# 智能体类
train, test	# 工作流函数
main	# 主函数

第七章 输出文件与日志

训练结束后，系统会在./logs/目录下生成以下文件：

- pong_model.zip：最终训练完成的模型
- best_model_{timestep}.zip：训练过程中奖励最高的模型
- env_config.pt：环境配置参数（用于测试时复现）
- PPO_Attention_*/：TensorBoard 日志文件夹
- aligned/：对齐步数后的 TensorBoard 日志（便于对比实验）